Australian National University

Lab Three

Seven-segment Displays and Switch Debouncers

Overview

Seven-segment displays provide an efficient way to display simple alpha-numerical information. In this lab, you will design a seven-segment display driver.

Mechanical buttons and switches sometimes *bounce* on their electrical contacts when pressed. In this lab, you will design a debouncer to make button and switch inputs more reliable.

In this lab, you will learn to:

- Instantiate modules in hierarchical designs.
- Use clocks, resets and enables to control the dynamic behaviour of your design.
- Create a display driver for the seven-segment display on the PMOD or the FPGA board.
- Improve the reliability of push button and switch inputs using a debouncer circuit.

When adding an existing file to a project, **we highly recommend that you check the box "copy sources (or constraints file) into project"**. This will create local copies of those files inside your new project folder.

If the option is not selected, Vivado will instead link to files at their original location. This risks those paths becoming corrupted if files are moved, and/or you may inadvertently modify files that belong to past projects, which could in turn cause those previous designs to no longer work.

Australian National University

Table of Contents

Learning Outcomes 3

Activity 1: Seven-segment Displays 3

Procedure 5

Step One: Create a new project in Vivado 5

Step Two: Add the heartbeat generator 5

Step Three: Create a counter that increases from 0 to 9 in 1 second increments 5

Step Four: Represent binary numbers as decimal digits on the SSD 6

Step Five: Select which display to illuminate 8

Step Six: Constrain you design 8

Step Seven: Test your design in hardware 9

Activity 2: Display driver for multiple SSDs 10

Procedure 11

Step One: Create a new project in Vivado 11

Step Two: Describe the behaviour of the display driver module 11

Step Three: Constrain your design and implement it on your board. 12

Activity 3: Switch debouncer (7 marks) 12

Procedure 15

Step One: Create a new project in Vivado 15

Step Two: Add given Verilog sources 15

Step Three: Create the debouncer 15

Step Four: Simulate the debouncer 16

Step Five: Create the single pulse on transition (SPOT) block 16

Step Six: Create the counter with enable and reset 16

Step Seven: Check the whole design and module interconnections 16

Step Eight: Constrain your design 16

Step Nine: Deploy the full design on your board 17

Appendix 1: using a Basys3 board for this lab 18

Activities 1 and 2 18

SSD configuration 18

Bus sizes 19

Hardware constraints 19

Behaviour of the display driver module 19

Activity 3 20

Design constraints 20

Clock speeds 20

 © ANU 2025

Learning Outcomes

By completing this lab, you will be able to:

1. Instantiate modules in hierarchical designs.
2. Display information using seven-segment displays.
3. Use synchronous reset and enable signals to control sequential logic processes.
4. Describe sequential logic circuits in Verilog including timers and shift-registers.
5. Design debouncers to improve the reliability of push button and switch inputs.

To complete this lab, you will require access to Vivado and the development Board (Zybo Z7 or Basys 3). Copies of the reference manuals are provided in Canvas.

Activity 1: Seven-segment Displays

[6 marks, approx. 30 min]

Seven-segment displays (SSDs) are simple and efficient electronic devices used to display alphanumeric characters. Seven-segment displays consist of seven segments that can be individually activated to display up to 2^7 = 128 different patterns.

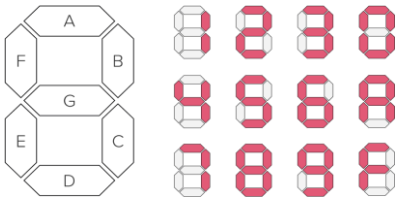

Figure 1: Illustration of a seven-segment display and some alphanumeric characters.

The Digilent boards used in this course use seven-segment LED displays. The Zybo Z7 does not have built-in displays, however, our lab kits include a 2-digit PMOD module which can be connected to the board as shown in Figure 2 below. Basys3 boards, on the other hand, are equipped with an on-board 4-digit display.

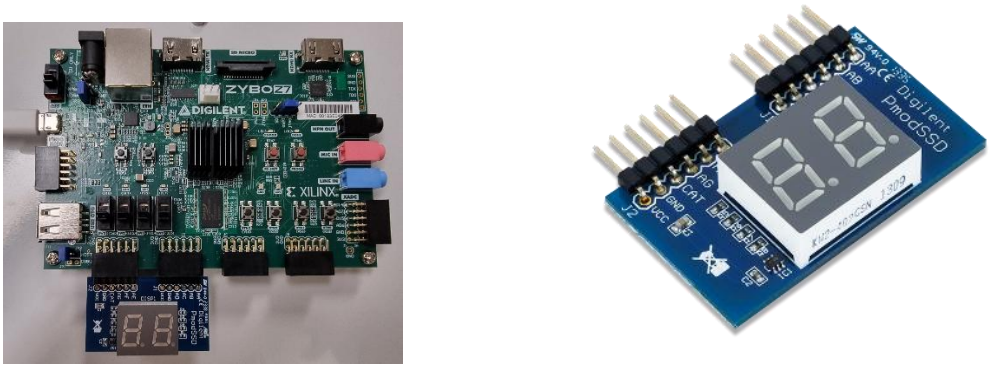

Figure 2: The Zybo Z7 board with attached PMOD SSD module (left), and a magnified image of the module (right).

© ANU 2025

Each of the SSD digits is composed of seven segments arranged as shown in Figure 3, with an LED embedded in each segment. Segment LEDs can be individually illuminated. Any one of the 128 patterns can be displayed on a digit by illuminating certain LED segments and leaving the others dark, with some examples shown in Figure 1.

To operate SSDs, it is essential to understand their electronic configuration. We explain here the operation of the PMOD SSD module, which utilises a *common-cathode* configuration. While the electrical principles are broadly similar, the Basys3 built-in SSDs use a *common-anode* configuration instead—interested students should referred to the Appendix for more detail.

LED segments light up when current traverses them, flowing from the anode terminal to the cathode terminal. In common-cathode displays, the cathode terminals of all seven LED segments forming each digit are tied together to a common circuit node, while the anodes remain separate (see Figure 3, left). Effectively, driving individual anodes high allows us to determine which segments will be lit up, while grounding the common cathode allows us to "enable" whole digits.

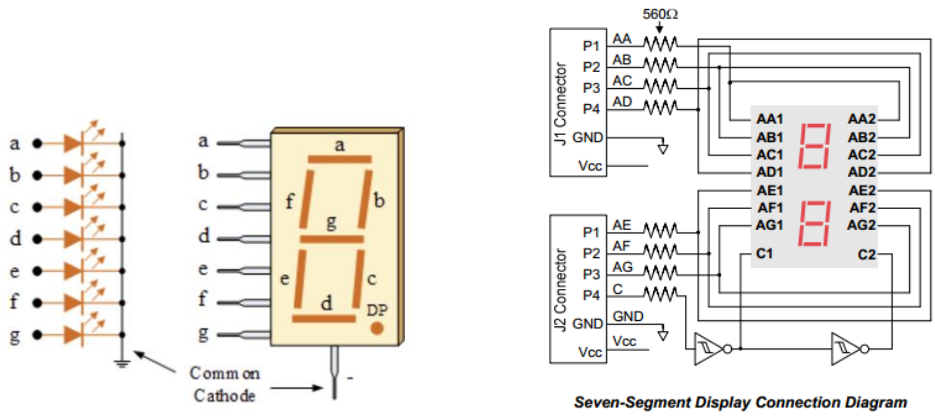

Figure 3: Common-cathode SSD configuration (left), and PMOD SSD electrical schematic (right).

Furthermore (see Figure 3, right), the anode circuit connections are shared amongst both digits. For example, input "AD" drives the anodes for the "d" segments in both digits, i.e., AD1 and AD2. That is why the PMOD SSD board only has just eight signal pins despite featuring two displays. Finally, note that the cathode line "C" (J2 connector, pin P4) is connected to both C1 and C2 but it features a pair of "trigger inverter" circuits along the way. These components are essentially fancy NOT gates, which force the levels seen at SSD inputs C1 and C2 to always be opposite. This means that *when "C" is driven HIGH, C1 is LOW (1st digit active) while C2 is HIGH (2nd digit is inactive), while the opposite happens when C1 is driven LOW*. This arrangement therefore always activates one display digit at a time, with "C" acting as the selector signal.

To display multiple different digits on the display, **we need to create a display driver module**, which takes care of two important tasks:

1. enabling the desired digit via cathode selection and simultaneously lighting up the relevant segments via anode activation.
2. where multiple digits need to be shown, driving the cathode "enable" signals and corresponding anode patterns in a repeating, continuous succession at an update rate that is

 © ANU 2025

fast enough to present to the human eye as though multiple digits are appearing simultaneously across the display.

Procedure

In this activity, we're going to display the value of a counter incremented every second. Note that the instructions are for PMOD SSD used in conjunction with a Zybo Z7 board. If you are using a Basys3 board, please refer to the Appendix.

Step One: Create a new project in Vivado

1. Create a new project 'single_seven_segment_display'.
2. Create a new design source 'singleSevenSegmentDisplay_TOP.v'.

   The inputs are `clk`, `reset`, and `enable`. The outputs are
   - A 7-bit wire called `ssdAnode`
   - A 1-bit wire called `ssdCathode`.

Step Two: Add the heartbeat generator

We want to increment the counter on a 1 Hz trigger signal. The heartbeat generator in **Lab 2, Activity 4** can deliver the trigger signal.

1. Add 'clockDividerHB.v' module from Lab 2, Activity 4 to the project. Set the `THRESHOLD` value to 62_500_000 (50_000_000 on the Basys3) to generate a 1 Hz divided clock and a 1 Hz heartbeat.

   The expression for the heartbeat is

   ```
   assign beat=(counter==THRESHOLD-1) & (dividedClk);
   ```

   The heartbeat is HIGH (i.e. `beat = 1`) for one 125MHz (100 MHz on Basys3) master clock cycle just before the divided clock switches from HIGH to LOW. If this is not clear, please add your clock divider test bench (used in Lab 2, Activity 4) as a simulation source and observe the simulation output.

Step Three: Create a counter that increases from 0 to 9 in 1 second increments

1. In the top-level module 'singleSevenSegmentDisplay_TOP.v', declare a 1-bit wire called `beat`. Instantiate clockDividerHB and set it to generate a 1 Hz beat. Since the output `dividedClk` is not used, you can leave the bracket empty (i.e., `.dividedClk()`).
2. Declare a 4-bit reg entity called `counter`. We are now ready to use the `beat` signal to increment this `counter` every second. This `counter` creates the sequence of digits which will be displayed on the seven-segment display.
3. Create a clocked process running on the master clock that will:
   a. Reset the `counter` to zero whenever the input reset signal is HIGH (`reset==1'b1`) or the `counter` value becomes greater than or equal to 10.

© ANU 2025

Optional: Consider why we are resetting this `counter` on a count of 10 and not 9. When the design is complete, test 9 instead of 10, see the result and try to figure out what might be happening.

b. Increment the `counter` when the heartbeat pulse is received (i.e., `beat==1'b1`) and do nothing when otherwise.

Step Four: Represent binary numbers as decimal digits on the SSD

Add source file 'sevenSegmentDecoder.v' from Canvas. The input is 4-bit wire `bcd` (the desired number—the acronym bcd stands for *binary-coded decimal*) and the output is 7-bit **reg** `ssd`, which represents the anode values. Go through the source file and note the following points:

1. Since `ssd` is updated inside a procedural `always` block, it is of type **reg.**
2. In Figure 3, each segment in the display is identified by an alphabetical character from A to G. All seven segments are combined into `ssd` with 'A' as its most-significant bit `ssd[6]` and 'G' as its least-significant bit `ssd[0]`. This allows us to code numbers as `ssd = 7'bABCDEFG` where ABCDEFG are zeros or ones representing which segments are switched ON or OFF. PMOD SSDs are **active high**, i.e., logic value 1 corresponds to ON. For instance, number 7 is `ssd = 7'b1110000`. (reminder: if using the Basys3, the display logic is *active low* instead – see Appendix)
3. The mapping from `bcd` to `ssd` is static (i.e., truth-table like), which means it can be designed as a decoder. Involving the clock in this part would be both unnecessary from a functional perspective and wasteful from a hardware recruitment perspective.

   There are two ways we can map `bcd` to `ssd`: 1) using a series of if / else statements; and 2) using case statements. Whilst both methods will do the same thing, they will be implemented in slightly different ways.

   If / else statements have implicit prioritisation based on the order and nesting of statements. The inherent priority of if / else statements is implemented using a '**priority routing network**', which is essentially a series of multiplexers connected to each other in series. Case structures, on the other hand, are implemented using one large multiplexer with a single input condition (the `case` argument).

   There are good reasons to use one over the other. For example, case structures are efficient when all possible input conditions are mutually exclusive (i.e., when only one condition can be true at a time), whereas if / else statements are more appropriate when more than a single input condition can be true at the same time, when prioritisation is important.

   Because all possible values of `bcd` are mutually exclusive, a **case** structure is appropriate.

   The following code describes the case structure for the mapping:

 © ANU 2025

```
always @(*) begin
    case(bcd)
        4'd0 : ssd = 7'b1111110;
        4'd1 : ssd = 7'b0110000;
        4'd2 : ssd = 7'b1101101;
        4'd3 : ssd = 7'b1111001;
        4'd4 : ssd = 7'b0110011;
        4'd5 : ssd = 7'b1011011;
        4'd6 : ssd = 7'b1011111;
        4'd7 : ssd = 7'b1110000;
        4'd8 : ssd = 7'b1111111;
        4'd9 : ssd = 7'b1111011;

        default : ssd = 7'b0000000;
    endcase
end
```

What this describes is a series of possible 'cases' for the signal `bcd`. It can be read as: "If `bcd == 4'd0`, then the value of `ssd` is `7'b1111110`". The `default` condition is essential because `bcd` could accidentally acquire a value that is not covered by the 'cases' mentioned (e.g., `4'd12`). Remember that combinational logic always needs to have a defined output for all possible input combinations.

A simplified diagram of the case structure is shown in Figure 4.

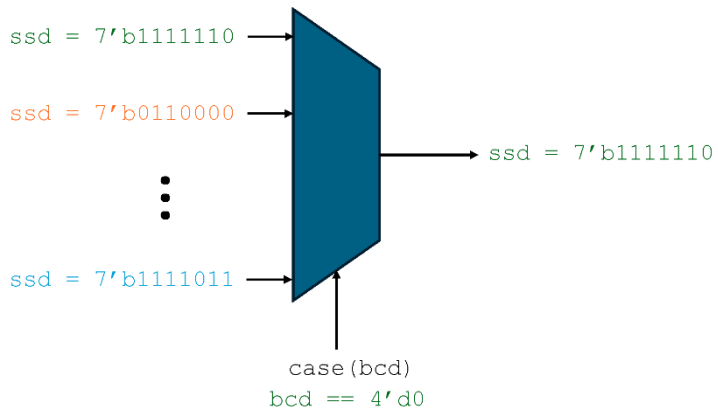

Figure 4: Simplified diagram of a case structure where a single large multiplexer is used to route different possible values of `ssd` to the output depending on the value of `bcd`.

Describing the same thing with a series of if / else statements would look like this:

© ANU 2025

```
always @(*) begin
    if (bcd == 4'd0) begin
          ssd = 7'b0000001;
     end else if (bcd == 4'd1) begin
          ssd = 7'b1001111;
    ...
     end else
          ssd = 7'b1111111;
     end
end
```

A simplified diagram of the if / else structure is shown in Figure 5.

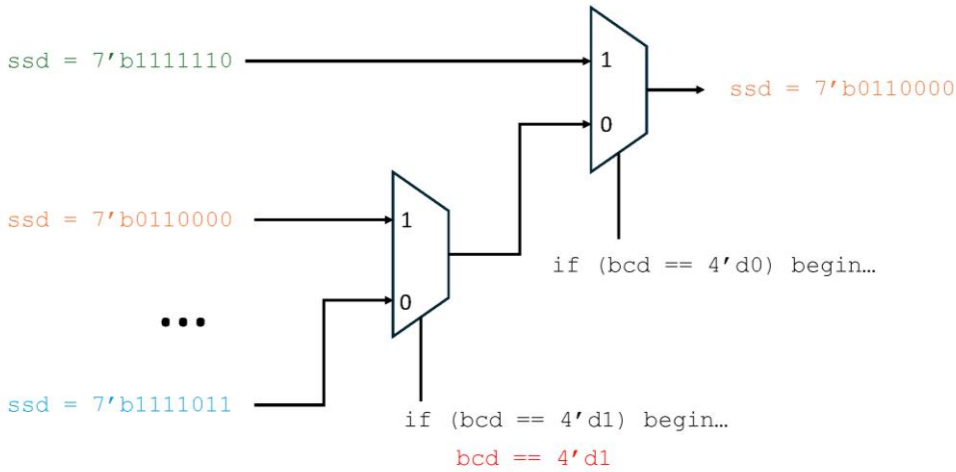

Figure 5: Simplified diagram of an if / else priority routed network. If the highest priority logic condition (bcd == 4'd0) is not satisfied, then the mux connects the output of the next highest priority condition to the output and so on.

4. Instantiate 'sevenSegmentDecoder.v' in the top-module 'singleSevenSegmentDisplay_TOP.v'. Connect the 4-bit `counter` in **Step Three** to the `bcd` input of 'sevenSegmentDecoder.v'. Connect the top-module's output signal `ssdAnode` directly to the `ssd` output of the instantiated 'sevenSegmentDecoder.v' module.

Step Five: Select which display to illuminate (For Zybo Z7 board. See Appendix for Basys3)

There are two seven-segment displays on the PMOD SSD. You can select which display to show the value of the `counter` by setting `ssdCathode` to 0 or 1. Continuously **assign** `ssdCathode` to be HIGH to display the counter on the left seven-segment display. (you can also choose to set it LOW, and see what happens!)

Step Six: Constrain you design

1. Create a constraints file 'singleSSD_constraints_ZyboZ7'.
2. Add constraints for `clk`, `reset` and `enable`.

© ANU 2025

3. Add constraints for the seven-segment displays' shared cathode (identified as C) and segment anodes (identified as AA to AG).

Your constraints file should look something like this (please note: some identifiers are missing!):

```
Set_property -dict {PACKAGE_PIN K17 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -name clock1 -add -period 8.00 [get_ports clk]

set_property -dict {PACKAGE_PIN K18 IOSTANDARD LVCMOS33} [get_ports reset]
set_property -dict {PACKAGE_PIN G15 IOSTANDARD LVCMOS33} [get_ports enable]

set_property -dict {PACKAGE_PIN H15 IOSTANDARD LVCMOS33} [get_ports {ssdCathode}]

set_property -dict {PACKAGE_PIN T14 IOSTANDARD LVCMOS33} [get_ports {ssdAnode[6]}]
set_property -dict {PACKAGE_PIN T15 IOSTANDARD LVCMOS33} [get_ports {ssdAnode[5]}]
set_property -dict {PACKAGE_PIN *?* IOSTANDARD LVCMOS33} [get_ports {ssdAnode[4]}]
set_property -dict {PACKAGE_PIN *?* IOSTANDARD LVCMOS33} [get_ports {ssdAnode[3]}]
set_property -dict {PACKAGE_PIN V12 IOSTANDARD LVCMOS33} [get_ports {ssdAnode[2]}]
set_property -dict {PACKAGE_PIN W16 IOSTANDARD LVCMOS33} [get_ports {ssdAnode[1]}]
set_property -dict {PACKAGE_PIN *?* IOSTANDARD LVCMOS33} [get_ports {ssdAnode[0]}]
```

Please note that any identifiers provided in **green** above assume that the PMOD SSD module is connected to ports JD and JE, as shown in Figure 2. If you plug your module into a different set of ports, you will need to use different identifiers.

Please also note that the seven-segment display anode numbers are assigned as [6] for AA (the anode pin for segment "a") and [0] for AG (the anode pin for segment "g"). This has been done for consistency with the `7'bABCDEFG` data structure chosen earlier for `ssd`.

Use the Z7 board reference manual to find the correct identifiers for the PMOD port pins. We discussed how to PMOD port pins are numbered in Lab 2 (see Figure 6 for a refresher). To know which anode/cathode signal line connects to which pin, you can refer to the pinout for the PMOD SSD module previously shown in Figure 3-right. The reference manual for the PMOD SSD module is on the Digilent website at https://digilent.com/reference/pmod/pmodssd/reference-manual.

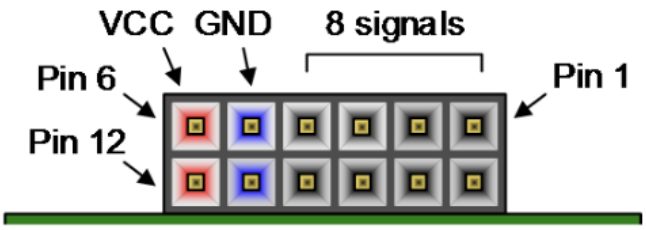

Figure 6: how PMOD pins are numbered on the Zybo Z7.

Step Seven: Test your design in hardware

1. Right click on the file 'singleSevenSegmentDisplay_TOP.v' in the Vivado sources browser window and select the 'set as top' option. This will ensure that Vivado interprets this file as the top-level module.
2. Check one more time that everything is connected correctly in your top module. This includes names of submodules, internal signal declarations, etc.

 © ANU 2025

3. Run Synthesis, Implementation and Generate Bitstream.

   If any step fails, then read through the Messages tab down the bottom for errors. They will give you some good hints as to what is going wrong.

4. Program the device. Apart from the seven-segment display, please also check the operation of enable and reset.

Please show your work to the tutor.

As they check your work, the tutors may wish to review the following in your notebook:

- A hand-drawn sketch of the RTL structure of your system, demonstrating your understanding of how heartbeat generator, counter, decoder and SSD work together in this design. You should not blindly mirror the Vivado RTL diagram: try to draw a diagram that makes good sense to you, free of distracting detail.

Activity 2: Display driver for multiple SSDs
[7 marks, approx. 30 min]

As mentioned earlier in Activity 1, an annoying thing about sharing the segment signals across multiple seven-segment displays is that we can't operate each digit independently. To display different information on each display, it is necessary to activate a single cathode (or anode, if working with the Basys3) at a time in rapid succession. With this approach, only a single display is illuminated at any time instant. If we cycle through the displays fast enough, then the human eye won't notice the scanning. We will call our module a *display driver*. The human eye can perceive periodic signals at up to about 60 Hz, so if we illuminate each display for 2 ms, then the array of two displays will have a total refresh rate of 250 Hz, which is fast enough for humans not to notice. A consequence of activating each display for a fraction of the time, as opposed to keeping them "always on", is that they may appear (slightly) less bright.

The timing diagram for the two-digit PMOD SSD module is shown in Figure 7 below.

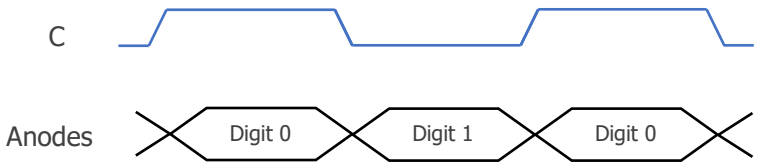

Figure 7: Timing diagram for the 2-digit PMOD SSD module.

In this activity, we will display '**37**' on the seven-segment displays.

 © ANU 2025

Procedure

Step One: Create a new project in Vivado

1. Create a new project 'multiple_seven_segment_displays'.
2. Create a new Verilog design source 'multipleSevenSegmentDisplays_TOP.v'.

   The input is `clk` and the outputs are

   - A 7-bit wire called `ssdAnode`
   - A 1-bit **reg** called `ssdCathode`.

Step Two: Describe the behaviour of the display driver module

1. To illuminate each seven-segment display for 2 ms, we need a 500 Hz heartbeat. Declare a 1-bit wire beat. Add and instantiate the 'clockDividerHB.v' module, setting THRESHOLD appropriately to generate a 500Hz heartbeat. We never want the heartbeat to be paused or reset, so set its enable input to be constantly `1'b1` and its reset to `1'b0`. We're going to change which display is currently active every time a heartbeat is received.
2. To achieve the toggling of the two displays, we can create a simple T-flip flop structure. Declare a single-bit **reg** called `activeDisplay` and describe its behaviour as a clocked process that inverts its output every time the 500Hz beat is asserted.
3. Finally, add a behavioural code block describing the behaviour of `ssdCathode` and `ssdAnode` for each case of `activeDisplay`. With two displays, signal `ssdCathode` will be a 0 or a 1. To easily determine what `ssdAnode` bit word is needed to display a desired number, we can leverage the decoder module we used in the previous activity. Declare a 4-bit **reg** `ssdValue`, which will represent the bcd encoding of the number to be displayed on the SSD.

   Since all possible values of `activeDisplay` are mutually exclusive, we're going to describe this behaviour using a '`case`' structure. Each case has multiple assignments. Therefore, we need to use `begin` and `end` keywords for each case:

```
always @(*) begin
         case(activeDisplay)
               1'd0 : begin
                     ssdValue = 4'd3; //1st digit of number '37'
                     ssdCathode = 1;
              end
              1'd1 : begin //**TO DO: add content of other case

               end

               default : begin
                    ssdValue    = 4'd10; // undefined - should not occur
                    ssdCathode = 0;
               end
         endcase
end
```


© ANU 2025

Since `ssdAnode` and `ssdValue` are updated inside a procedural block, they are entities of type **reg**.

4. The SSD is expected to show **37**. In the code above, the first digit is '3' when `activeDisplay == 1'd0`, while the second case was left for you to complete.

   Note: as we only have two digits, the behaviour of `ssdCathode` may seem trivial here (indeed, it is) and you may wonder why we bothered defining `activeDisplay` and why we used a `case` structure, instead of a simple `if/else`. This is a fair question. Indeed, simpler code would have sufficed here. However, the slightly more generalised structure of this module allows it to be adapted to situations where you may need to drive more than 2 displays, possibly with a more complex "digit enable" logic. To better appreciate this, you could try to create a driver for all *four* digits on the Basys3 (see Appendix for hints).

5. Instantiate the module 'sevenSegmentDecoder.v' in the top-module 'multiSevenSegmentDisplays_TOP.v'. Connect `ssdValue` to the input of 'sevenSegmentDecoder.v' and `ssdAnode` to the output of 'sevenSegmentDecoder.v'.

Step Three: Constrain your design and implement it on your board.

1. Create 'multipleSSD_constraints_ZyboZ7.xdc'. The constraints file is similar to that used in Activity 1, so you can import your previous file and modify it as required.
2. Run Synthesis, Implementation and Generate Bitstream. Program the device and check that '**37**' is shown correctly on the seven-segment displays.

Please show your work to the tutor.

As they check your work, the tutors will want to review the following in your notebook:

- A hand-drawn sketch of the RTL structure of your system, demonstrating your understanding of how heartbeat generator, display selector, decoder and SSD work together in this design. You should not blindly mirror the Vivado RTL diagram: try to draw a diagram that makes good sense to you, free of distracting detail.

Activity 3: Switch debouncer (7 marks)
[7 marks, approx. 1h 30min]

Mechanical buttons and switches sometimes *bounce* on their electrical contacts when pressed, generating what is called *switch chatter*. In many circumstances, bouncing leads to undesirable behaviour. For example, imagine pressing the 'S' key on your keyboard a single time and having the letter appear on your screen three or four times. Therefore, practically every mechanical button or switch has a debouncer of some form or another.

 © ANU 2025

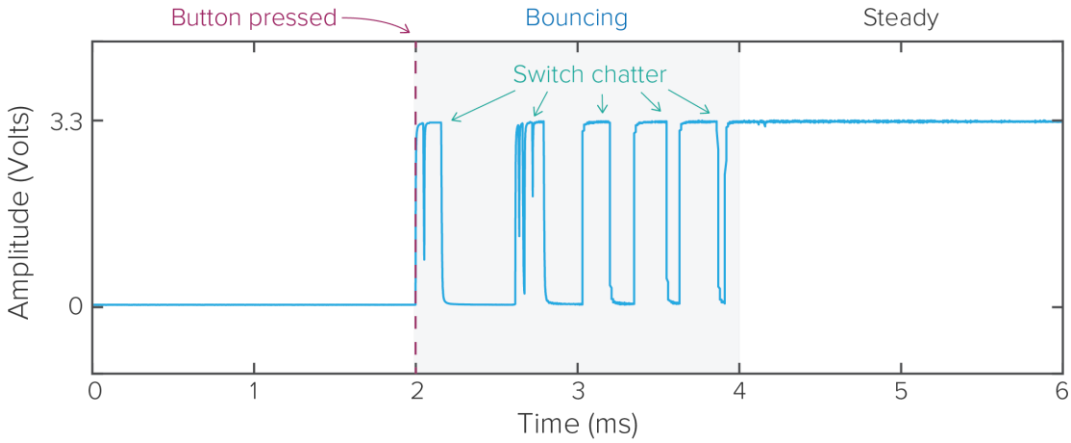

Figure 8: Measurement of switch chatter produced by 'bouncing' electrical contacts of a mechanical switch.

There are many ways to implement a debouncer in both hardware (e.g., placing a simple RC low-pass filter circuit on the output of a button or switch), and in software (e.g., designing an algorithm that counts how many times a button bounces in a specific period of time). In this activity, we're going to design a simple debouncer solution based on the below synchronous sequential design.

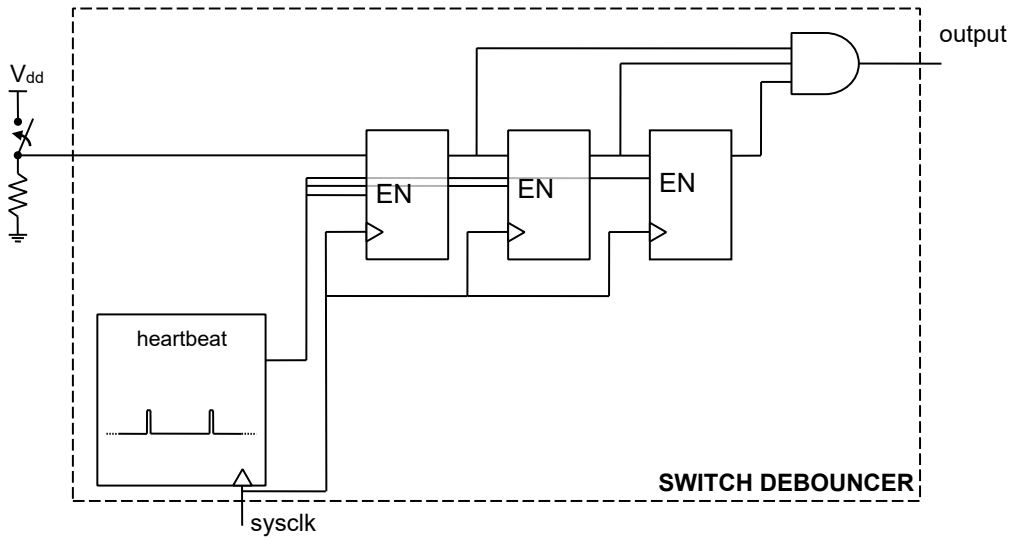

Figure 9: Block diagram description of a debouncer design.

The principle behind this system is that a valid input is recognised only if the input signal remains asserted (HIGH) for at least three consecutive time intervals of appropriate time length (the serial connection of D flip-flops creates a shift register). The debounced output is only HIGH when all elements in the shift-register are HIGH. EN means enable.

Common questions when designing a debouncer relate to the number of elements the shift-register needs to have, and the frequency at which the input signal needs to be sampled. The answers to these come down to a trade-off between how well the debouncer filters out switch chatter and the time it takes for the input to be recognised after being physically switched. This *latency* associated with the switch debouncer is approximately equal to the total propagation delay through the shift-register pipeline plus the duration of the bouncing. For example, in the absence of any switch chatter, a 4-bit shift-register pipeline sampling the input signal at 1 kHz could have

 © ANU 2025

a latency of up to 4 milliseconds depending on how it is designed. If the button or switch bounces for 1.3 milliseconds, then the total time it will take for the button press to be registered could be up to 5.3 milliseconds. This may be unacceptable in some applications, which is why it is necessary to understand the specific requirements for your system.

In the following exercise you will create your own switch debouncer and we will check that it works by using it as the input to a counter (the counter will operate as an event counter).

We will implement the circuit summarised in Figure 10 and this will allow us to compare the behaviour of a debounced and a non-debounced switch. Flicking either of the switches should cause the counter to count up, while pressing the rightmost button will reset the counter to zero. LEDs show the value of the count (in binary form). Make sure you understand the role of the *synchroniser* and the dotted box identified as *Single pulse on transition (SPOT)*.

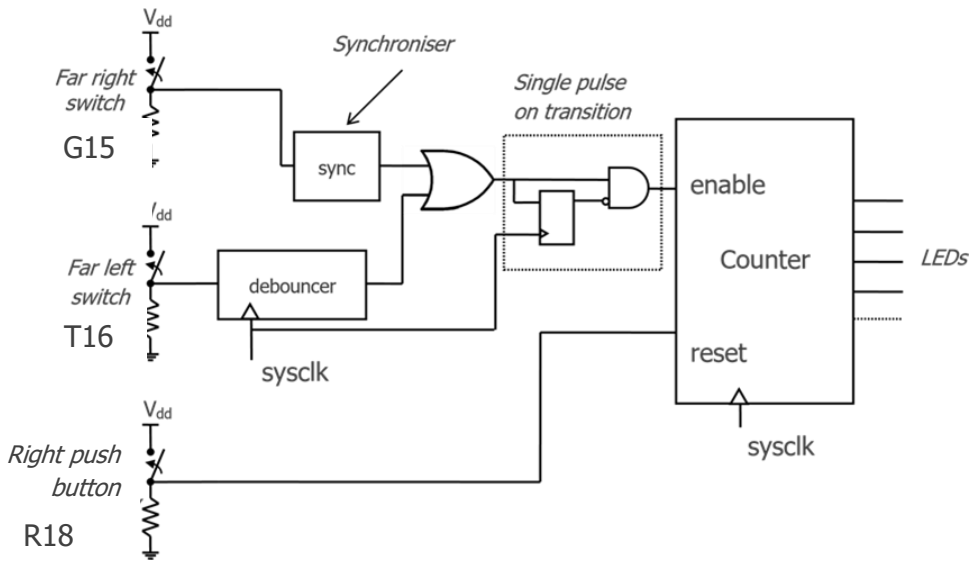

Figure 10: Block diagram for the debouncer testing exercise..

- The synchroniser (given as a file 'sync.v' as part of the lab resources) is essentially a chain of flipflops. It ensures that an external asynchronous input is clock-sampled and thus compliant with the timing requirements (e.g., set-up or hold times) of the deeper parts of the logic circuit, to reduce the risk of metastability incidents.

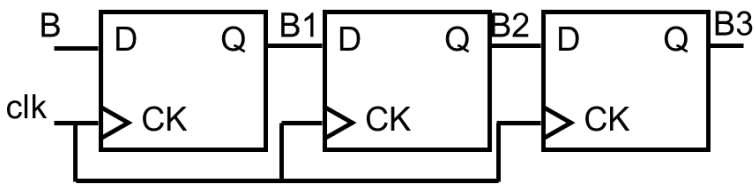

Figure 11: A synchroniser.

- The SPOT (Single Pulse On Transition) circuit ensures that, as the output of the OR block changes from 0 to 1 (switch event), the output of the AND block will also switch to 1, but **only for one system clock cycle** (the output of the flip-flop will switch to 1 and cause the AND block output to return to 0). **Each transition of the SPOT input from 0 to 1 will be therefore associated with a single output pulse of 1 clock cycle duration.**

 © ANU 2025

For this design, you will be supplied with pre-written design source for several blocks. It is recommended that you take some time scrutinising each supplied file to make sure you can fully understand its content.

Procedure

Step One: Create a new project in Vivado

Create a new project in Vivado called 'debouncetest'.

Step Two: Add given Verilog sources

1. Add the given design files for the project. In the resources given you should find:
    a. 'debouncetest_top.v'. This will be the top module of your design. It contains the module declaration and the instantiations of submodules. It should be ready to go but you should double-check it prior to the final implementation.
    b. 'sync.v'. This is ready to go and implements the synchroniser from Figure 10.
    c. 'counter.v'. This file contains the module declaration only and you will need to implement a 4-bit counter with enable and reset.
    d. 'spot.v'. This file contains the module declaration only and you will need to write the rest of the code, based on the diagram shown in Figure 11.
    e. 'debouncer.v'. This file contains the module declaration only and you will need to write the rest of the code, based on the diagram shown in Figure 9.
2. Also add the 'clockDividerHB.v' into the project, to help you drive heartbeats where needed.

Step Three: Create the debouncer

1. Open the 'debouncer.v' file for editing
2. In the code, instantiate clockDividerHB. Choose a `THRESHOLD` value of `4_166_667`, which will give you an adequate heartbeat frequency so that the debouncer returns a logic level 1 after the switch is ON for approximately 0.2s. Set enable to constant 1'b1.
3. Write Verilog code to describe the heartbeat-enabled shift register.

    We want the shift-register `pipeline` to push its data forward each time the register chain is enabled by the `beat` signal. This is easily done through a clocked process.

```
reg[2:0] pipeline;
always @(posedge clk) begin
      if (beat) begin
            pipeline[0] <= switchIn;
            pipeline[1] <= pipeline[0];
            pipeline[2] <= pipeline[1];
      end
end
```


© ANU 2025

4. Create the output logic. This is a combinatorial circuit combining the output of all flipflops in the shift register.

```
assign debounceout = &pipeline;
```

The syntax `&pipeline` is unique to Verilog and means 'all bits of pipeline being 1'; essentially an AND operation between `pipeline[0]`, `pipeline[1]` and `pipeline[2]`.

Step Four: Simulate the debouncer

Add 'TEST_debouncer.v' as a simulation source.

Set the THRESHOLD of clockDividerHB to 4_167. Run your simulation and check for correct operation. After simulation, remember to revert to the higher `THRESHOLD` value 4_166_667 before synthesis.

SIMULATION TIP: If signals associated to registers appear unexpectedly as X (undefined) in a simulation output, this may be due to a failure to initialise the registers. This is not usually required for hardware operation; however, it does pop up from time to time in simulations. Essentially, since the simulator does not know the initial value of a register, it fails to deliver a defined result for subsequent operations (e.g., "unknown" + 1 = "unknown"!). To solve this, your design includes a reset which forces registers to a known state, ensure that you trigger that reset as part of your testbench signals.

Step Five: Create the single pulse on transition (SPOT) block

Open the supplied 'spot.v' source for editing. Add your own code to implement the SPOT section of the circuit from Figure 10.

Hints: the code is simple. You will just need one short sequential always block to infer the D-flip-flop, followed by a continuous assignment to generate the gated output.

Step Six: Create the counter with enable and reset

Open 'counter.v'. Implement the 4-bit counter with enable and reset.

Step Seven: Check the whole design and module interconnections

The design should now be complete. Before we continue to implementation, it is best to do a check of the overall design. To do this, start from the top module 'debouncetest_top.v' and dig down into each submodule.

Have a look at the elaborated RTL schematic that Vivado generates. Can you match it with your design intent?

Step Eight: Constrain your design

We need to create a clock, connect our design up to the two slide switches and the centre push button, and assign the counter output to a few LEDs, so that we can view the count result. Double check that the signal names listed after `get_ports` match the names of the signals in your design.

 © ANU 2025

The constraints for slide switches and reset button:

```
set_property -dict {PACKAGE_PIN T16 IOSTANDARD LVCMOS33} [get_ports switchA]
set_property -dict {PACKAGE_PIN G15 IOSTANDARD LVCMOS33} [get_ports switchB]
set_property -dict {PACKAGE_PIN R18 IOSTANDARD LVCMOS33} [get_ports reset]
```

The constraints for LEDs:

```
set_property -dict {PACKAGE_PIN D18 IOSTANDARD LVCMOS33} [get_ports {led[3]}]
set_property -dict {PACKAGE_PIN G14 IOSTANDARD LVCMOS33} [get_ports {led[2]}]
set_property -dict {PACKAGE_PIN M15 IOSTANDARD LVCMOS33} [get_ports {led[1]}]
set_property -dict {PACKAGE_PIN M14 IOSTANDARD LVCMOS33} [get_ports {led[0]}]
```

The constraints for `clk`:

```
set_property -dict {PACKAGE_PIN K17 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -name clock1 -add -period 8.00 [get_ports clk]
```

Step Nine: Deploy the full design on your board

1. Run Synthesis, Implementation and Generate Bitstream.
2. Check the operation of your design.
   a. Do you see a difference in the count behaviour between the two switches? Can you explain it considering the operation of the system?

      Tip: if you are struggling to see a difference, try to operate the switches in different ways (faster, slower, etc.) to try and generate the bouncing events in the middle of the transition.

   b. Do you notice a difference in terms of delay in the count update between the debounced and non-debounced switches? Can you switch the debounced switch so quickly that a count does not register? What does that tell you about trade-offs between performance and accuracy?

Give your tutor a demonstration and explanation of your results.

As they check your work, the tutors may wish to review the following in your notebook:

- A sketch of the internal signal traces of the debouncer, the synchroniser, and the spot circuits, so that you can demonstrate understanding and explain in your own words how each works.
- Your best explanation of why the sync circuit does not debounce the button, but the debouncer circuit does.
- Notes on observations made while experimenting with the hardware implementation (as per Step Nine)

End of Lab 3

(Keep scrolling for Appendices)

 © ANU 2025

Appendix 1: using a Basys3 board for this lab

Most of the activities in this manual can be attempted by students at home on a Basys3 board as well. Here are some of the key aspects to keep in mind when translating across the two devices:

Activities 1 and 2

SSD configuration

The Basys3 runs a 4-digit screen with **common-anode configuration**. A common-anode SSD is configured with opposite principles to the common-cathode case described in the main document. Anode lines "AN1…4" act as *digit enable*, while cathode lines "CA…CG" *light up the corresponding segment* on active digits.

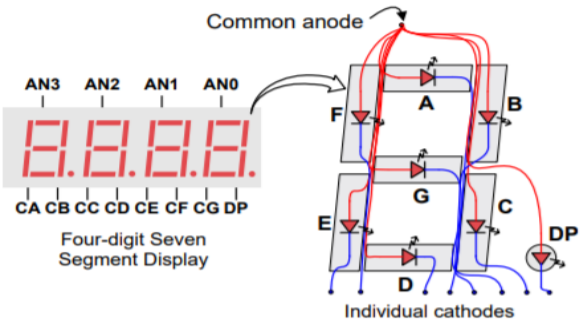

Figure A1: Circuit configuration of the 4-digit seven-segment display on the Basys3. AN stands for anode. CA to CG are the seven segments. DP is the decimal point.

To illuminate a segment, the anode should be typically driven high while the cathode is driven low. However, the Basys3 circuitry includes some inverting transistors, so logic levels are inverted. Therefore, **on the Basys3 board, anodes AN0 to AN3, cathodes CA to CG and DP are all driven low when active.** A key implication for this is that the outputs of the bcd-to-ssd decoder used in the ZyboZ7 design will need to be negated in order to work properly in the Basys3. To display a multi-digit expression across four digits, the display driver for the Basys3 will need to be designed to output a periodic, rapidly switching anode enable sequence (0111->1011->1101->1110) and, upon every digit switch, match the cathode signals to the desired pattern.

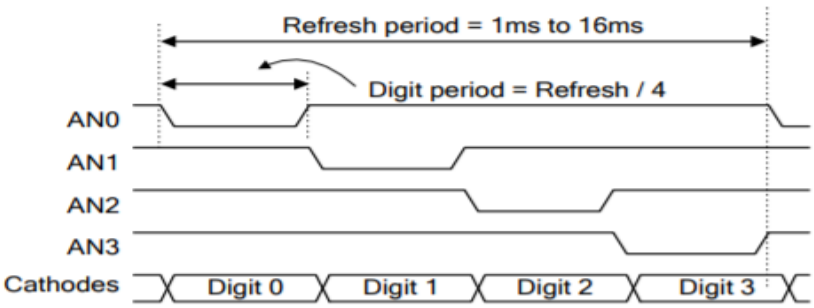

Figure A2: Timing diagram of the display driver module for the Basys3 4-digit seven-segment display.

© ANU 2025

Note: unlike the PMOD SSD, the wiring of the Basys3 SSDs makes it possible to have multiple digits active at the same time. You could, for example, run a design with multiple anode pins activated at once, e.g., `4'b0110`. Note that digits that are enabled simultaneously will also display the same pattern.

Bus sizes

Though implied in the descriptions provided earlier, for a Basys3 board we would define signals `ssdCathode` as a 7-bit bus (segment select), and `ssdAnode` as a 4-bit bus (digit select).

Hardware constraints

The constraints file for the SSD exercises on the Basys3 will use different pin locations to that of the Z7. Here it is for your reference. This constraint example uses the centre push button as the reset button.

```
set_property -dict {PACKAGE_PIN W5 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -name clock1 -add -period 10.00 [get_ports clk]

set_property -dict {PACKAGE_PIN U18 IOSTANDARD LVCMOS33} [get_ports reset]
set_property -dict {PACKAGE_PIN V17 IOSTANDARD LVCMOS33} [get_ports enable]

set_property -dict {PACKAGE_PIN U2 IOSTANDARD LVCMOS33} [get_ports {ssdAnode[0]}]
set_property -dict {PACKAGE_PIN U4 IOSTANDARD LVCMOS33} [get_ports {ssdAnode[1]}]
set_property -dict {PACKAGE_PIN V4 IOSTANDARD LVCMOS33} [get_ports {ssdAnode[2]}]
set_property -dict {PACKAGE_PIN W4 IOSTANDARD LVCMOS33} [get_ports {ssdAnode[3]}]

set_property -dict {PACKAGE_PIN W7 IOSTANDARD LVCMOS33} [get_ports {ssdCathode[6]}]
set_property -dict {PACKAGE_PIN W6 IOSTANDARD LVCMOS33} [get_ports {ssdCathode[5]}]
set_property -dict {PACKAGE_PIN U8 IOSTANDARD LVCMOS33} [get_ports {ssdCathode[4]}]
set_property -dict {PACKAGE_PIN V8 IOSTANDARD LVCMOS33} [get_ports {ssdCathode[3]}]
set_property -dict {PACKAGE_PIN U5 IOSTANDARD LVCMOS33} [get_ports {ssdCathode[2]}]
set_property -dict {PACKAGE_PIN V5 IOSTANDARD LVCMOS33} [get_ports {ssdCathode[1]}]
set_property -dict {PACKAGE_PIN U7 IOSTANDARD LVCMOS33} [get_ports {ssdCathode[0]}]
```

Please note that the display cathode numbers are assigned as [6] for CA and [0] for CG. This has been done for consistency with the `7'bABCDEFG` data structure chosen earlier for `ssd`.

Behaviour of the display driver module

There are four seven-segment displays on the Basys3. You can solve the exercise with just two (similar to the lab activity) or go for all four as an extension.

If you want to just use two SSDs, follow the same instructions used in the lab activity and create a single-bit activeDisplay reg entity, clocked at 500Hz. The key difference will be in the way the output case statement encodes the "digit select" signal (ssdAnode here). This will need to be an appropriately defined 2-bit signal (instead of being a single-bit signal, as was the case for the the PMOD SSD).

If you want to try using all four SSDs, a multi-bit signal for digit selection is required. A simple way to do this is to use a 2-bit overflow counter (counting 0 to 3 over and over again). If you choose this approach, `activeDisplay` will be a 2-bit reg entity. To keep the 250Hz refresh rate, the counter will increment with a 1 kHz beat. The output case statement will now have four cases. Try to visualise number "**4537**".

© ANU 2025

Activity 3

Activity 3 can be completed on both platforms with minimal differences. The main considerations in translating the Activity 3 design between Zybo Z7 and Basys3 boards relate to the identifiers of the I/O locations, and the different clock speed of the two boards.

Design constraints

Here are given constraints to create a clock, connect the design up to two slide switches and the centre push button, and assign the counter output to a few LEDs, so that you can view the count result. Double check that the signal names listed after `get_ports` match the names of the signals in your design, and ensure that you have visually identified the corresponding i/o hardware locations on the board.

The constraints for slide switches and reset button:

```
set_property -dict {PACKAGE_PIN R2 IOSTANDARD LVCMOS33} [get_ports switchA]
set_property -dict {PACKAGE_PIN V17 IOSTANDARD LVCMOS33} [get_ports switchB]
set_property -dict {PACKAGE_PIN U18 IOSTANDARD LVCMOS33} [get_ports reset]
```

The constraints for LEDs:

```
set_property -dict {PACKAGE_PIN L1 IOSTANDARD LVCMOS33} [get_ports {led[3]}]
set_property -dict {PACKAGE_PIN P1 IOSTANDARD LVCMOS33} [get_ports {led[2]}]
set_property -dict {PACKAGE_PIN N3 IOSTANDARD LVCMOS33} [get_ports {led[1]}]
set_property -dict {PACKAGE_PIN P3 IOSTANDARD LVCMOS33} [get_ports {led[0]}]
```

The constraints for `clk`:

```
set_property -dict {PACKAGE_PIN W5 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -name clock1 -add -period 10.00 [get_ports clk]
```

Clock speeds

The Basys3 runs at 100MHz, so the clockdivider settings to set the timing of the debouncer may need to be changed. To have a ~0.2s switch debouncer with the given design on a 100MHz clock, the HB threshold should be set at 3_333_333.

 © ANU 2025